CS & IT ENGINEERING



Algorithms

Analysis of Algorithms

Lecture No.- 12



By- Aditya sir

Recap of Previous Lecture



Topic Time complexity of recursive Algo.

Topic Loop complexity

Topics to be Covered



Topic Loop complexities Advanced

Topic Space complexity



ABOUT ME

Hello, I'm Aditya.

1. Represented college as the first Google DSC Ambassador.
2. The only student from the batch to secure an internship at Amazon.
3. Appeared for GATE during BTech and secured AIR 60 in GATE in very first attempt - City topper
4. Had offer from IIT Bombay and IISc Bangalore to join the Masters program
5. Joined IIT Bombay for my 2 year Masters program in Data Science
6. Published multiple research papers in well known conferences along with the team
7. Received the prestigious excellence in Research award from IIT Bombay for my Masters thesis8. Completed my Masters with an overall GPA of 9.36/108. Joined Dream11 as a Data Scientist.9. Have mentored around 10k working professions in field of Data Science and Analytics10. Have been mentoring GATE aspirants to secure a great rank in limited time.11. Have got around 27.5K followers on Linkedin where I share my insights and guide students and professionals.



By- Aditya sir



Topic : Analysis of Algorithms

Nested loops complexity

Loop within loop

2 types:-

1. Nested dependent loops (Tricky)
2. Nested independent loops ☆ (Easy)



Topic : Analysis of Algorithms

Eg.1.

```
a = 0
for (i = 1; i <=n; i++)
{
    for (j =1; j <=n; j++)
    {
        a = a + 1
    }
}
```

for (i = 1; i <=n; i++) ⟶ O(n)

i = 1 ⟶ j : 1 → n : O(n)

i = 2 ⟶ j : 1 → n : O(n)

i = n ⟶ O(n)

Independent

O(n) × O(n)

= O(n²)



Topic : Analysis of Algorithms

Mutually Exclusive loops:

(Not nested)

```
Algo AJ(n, m)
{
    for (i =1; i <=n ; i++)
    {
        print (i)
    }
    for (j = 1; j <=m; j++)
    {
        print (j)
    }
}
```

$O(n)$

$O(m)$

$O(m+n)$

$= O(\max(m, n))$



Topic : Analysis of Algorithms

```
Algo AJ(n)
{
    for (i=1; i<=n; i++)
    {
        print (1)
    }

    for (i =1; i<=n; i++)
    {
        print (i+1)
    }

    for ((i=1; i<=n;i++)
        for (j =1, j <=n/5; j++)
            for (k =1; k <=n; k++)
            {
                break
            }
}
```

O(n)

O(n)

O(n²)

n

n/5

1 time

$O(n+n+n^2) = O(n^2)$

A	O(n)
B	$O(n^2)$ → 34%
C	$O(n^3)$ → 46%
D	$O(n^4)$



Topic : Analysis of Algorithms

1.
```
For (i =1; i<=n; i =i+1)
{
    print(i)
}
```
n

runs?

1 → 2 → 3 - - - - → n

n/1

2.
```
For (i=1; i<=n; i=i+2)
{
    print(i)
}
```
1 → 3 → 5 → 7 - - - → n

n/2

3.
```
For (i=1; i<=n; i +7)
{
    print(i)
}
```
1 → 8 → 15 → 22 - - - - → n

n/7

O(n)



Topic : Analysis of Algorithms

Generalized from

```
For (i=1; i<=n; i =i+b)
{
        print(i)
}
```

→ runs n/b times

$TC: O(n/b) = O(n)$



Topic : Analysis of Algorithms

1. For (i=1; i<=n; i=i*2)
   {
   print(i)
   }

$TC = O(\log_2 n)$

Way-1

i=1, $1*2=2^1$, 2^2, 2^3 ..... 2^k

Let assume loop runs for k form

$2^k = n$ (for last iteration)

$\boxed{k = \log_2 n}$

Way-2

Let n = 16

$i \Rightarrow \underbrace{2,4,8,16}_{4 times}$

$= \log_2 16$

$= 4$



Topic : Analysis of Algorithms

2. For (i=1; i<=n; i=i×5)

```
{
    print (i)
}
```

$\longrightarrow TC = O(\log_5 n)$

$i = 1, 5^1, 5^2 \ldots\ldots 5^k$

k^{th} iteration , , $5^k = n$

(last) $k = \log_5 n$

(iter)



Topic : Analysis of Algorithms

Generalized from

```
for (i=1; i<=n;i=i*b)
{
    print (i)
}
```

$TC = O(\log n)$

$1 \rightarrow b \rightarrow b^2 \rightarrow b^3 \cdots b^k$

$b^k = n$

$k = \log_b n$

$TC = O(\log_b n)$



Topic : Analysis of Algorithms

#Q. Algo AJ(n)

```
{
    for (j=1, j <= n/2 ; j++)
    {
        print(j)
    }

    i=0
    while (i<=n)
    {
        print(i)
        i=i+3
    }
    AJ2(n)
}
```

$O(n/2) \rightarrow O(n)$

$O(n/3) \rightarrow O(n)$

$AJ2(n) \rightarrow O(\log n)$

81%

- A $O(n)$
- B $O(\log_3 n)$
- C $O(n^2)$
- D $O(n\log_3 n)$



Topic : Analysis of Algorithms

PYQ

#Q.
```
For(i=1; i<=n; i++)          ⟶ O(n)
{
    for (j=1;j<=n; j++)      ⟶ O(n)
    {
        for(k=n/2 ; k<=n; k=k+n/2)
        {
            print("AJ")
        }
    }
}
```

2 times

$n/2 + n/2 = n$

$n + n/2 \neq n$

$\Rightarrow TC: O(n \times n \times 2)$

$= O(n^2)$

A. $O(n)$

B. $O(n^2)$ ⟶ 46%

C. $O(n^3)$ ⟶ 32%

D. $O(n^2 \log n)$



Topic : Analysis of Algorithms

6.

```
i=1
while (I <=n)
{
        i=i*5
            print (i)
}
```

$\Rightarrow O(\log_5 n)$



Topic : Analysis of Algorithms

7.

```
J = n
while (j >0)
{
    j = j/5
        print(j)
}
```

$j = n, n/5, n/5^2, n/5^3 \ldots n/5^k$

$O(\log_5 n)$

$j = n \rightarrow n/5 \rightarrow n/5^2 \rightarrow n/5^3 \cdots n/5^k$ (k)

$n/5^k = 1$

$n = 5^k$

$(k = \log_5 n)$



Topic : Analysis of Algorithms

8.

```
C =0
for (i=1; i<=n; i=i+2)
for (j=1; j<=m; j=j*2)
{
    c = c+1
}
```

$O(n/2)$

$O(\log_2 m)$

$(n/2 * \log_2 m) * 1$

If $m = 2^n$

then value of c in terms of n ?

$n/2 \times \log_2(2^n)$

$= n/2 \times n = \boxed{\frac{n^2}{2}}$



Topic : Analysis of Algorithms

#Q.

```
C = 0
for (i=1; i< =n; i++)
{
    for (j=i,j<=n; j++)
    {
        c = c+1
    }
}
```

O(n)

Dependent

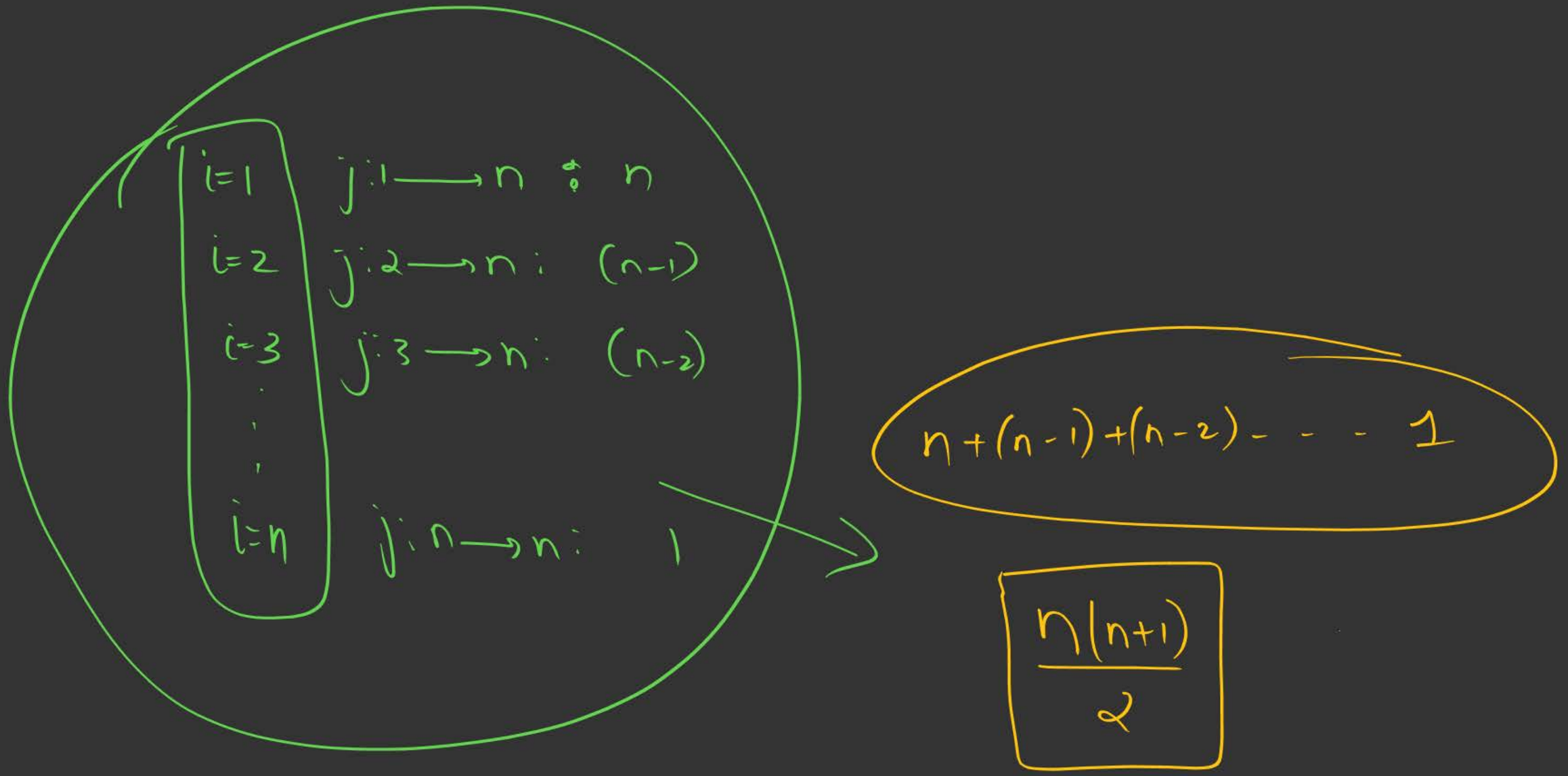
67%

A. $O(n)$

B. $O(n^3)$

C. $O(n \log n)$

D. $O(n^2)$


i=1 j: 1 ⟶ n : n
i=2 j: 2 ⟶ n : (n-1)
i=3 j: 3 ⟶ n : (n-2)
i=n j: n ⟶ n : 1
n+(n-1)+(n-2) - - - 1
n(n+1)/2



Topic : Analysis of Algorithms

#Q.

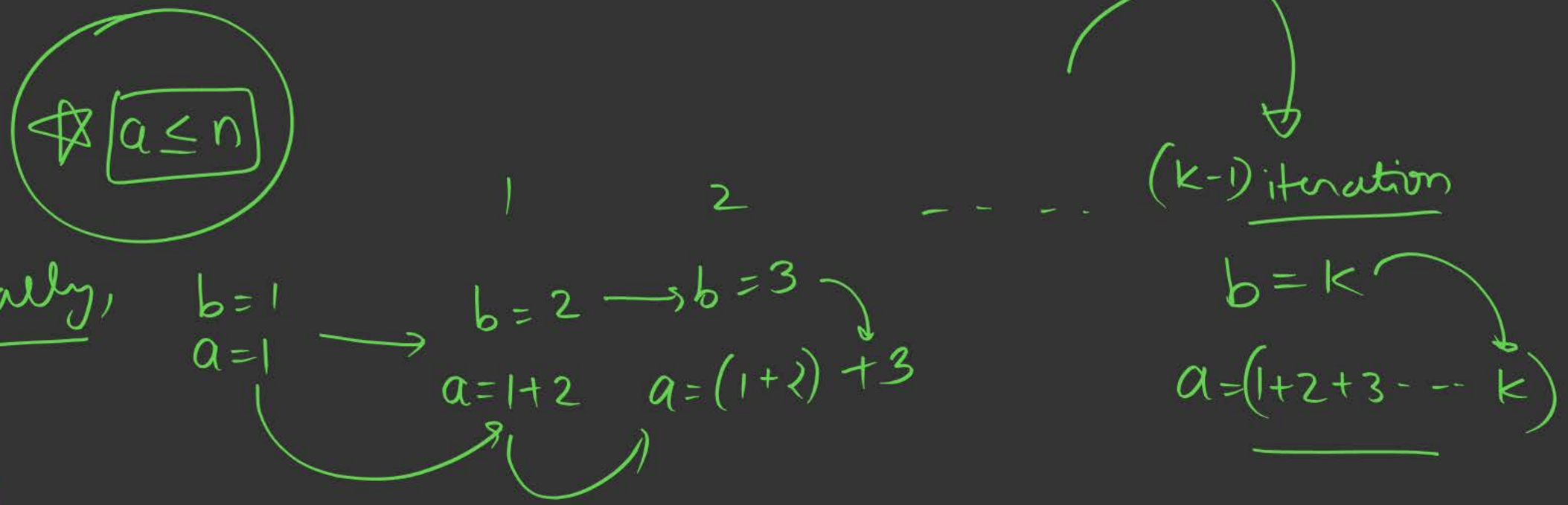
```
a = 1, b =1
while (a<=n)
{
    b = b +1
    a = a + b
}
```

A $O(n)$

B $O(\sqrt{n})$ → 2차

C $O(n^2)$

D $O(\log n)$



$b = b+1$

$a = a+b$

after k iter,

$a = 1+2+3 \cdots k$

$= \frac{k(k+1)}{2}$

last iter

$a \longrightarrow n$

$\frac{k(k+1)}{2} = n$

$O(k^2) = O(n)$

$k = O(\sqrt{n})$

$k^2 = a \times n$

$k = \sqrt{a \times n}$

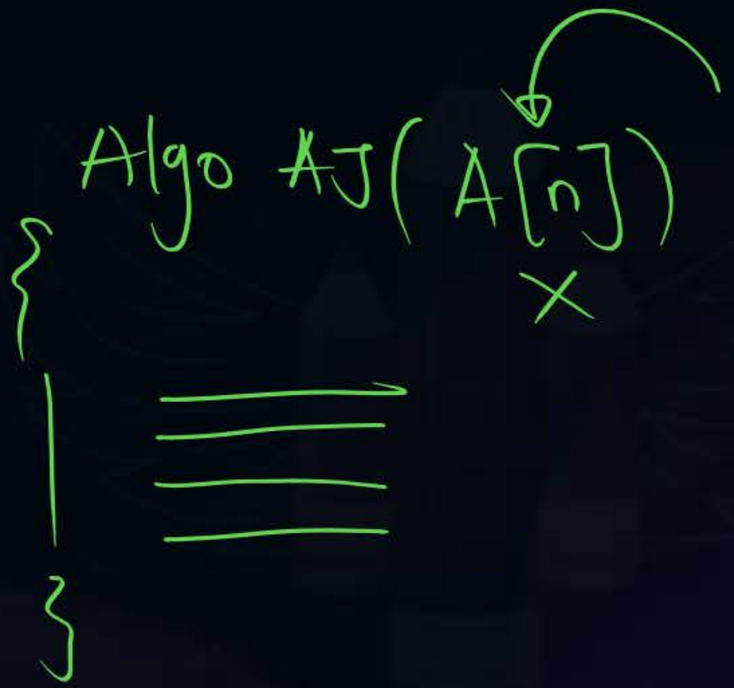
$k = O(\sqrt{n})$

Space Complexity !



Topic : Space Complexity

We define the space used by an algorithm to be the number of memory calls (or words) needed to carry out the computation steps required to solve an instance of the problem excluding the space allocated to hold the input. In other word, it is only the work space required by the algorithm





Topic : Space Complexity

```
Algo (input)
{
        ..............
}
```

Space complexity

Auxiliary space

(Additional space except the input)



Topic : Space Complexity

Eg.3. Linear search

```
Algo AJ Search(A[n] , x
{
    for (i=1; i<=n; i++)
        if (A[i] == x)
            return i
}
```

TC → O(n)

SC → var i → O(1)



Topic : Space Complexity

Eg.4. Algo sum AJ (A[n] , n)

```
{
    int sum = 0
        for (i = 1; i<=n; i++)
        {
            sum = sum +A[i]
        }
}
```

Var (sum , i)

SC : O(1)



Topic : Space Complexity

Eg.4. Algo recursive sum (A, n)

```
{
        it (n ==1)
            return A[n]
        else
            return (A[n] + recursive sum (A, n-1)
}
```



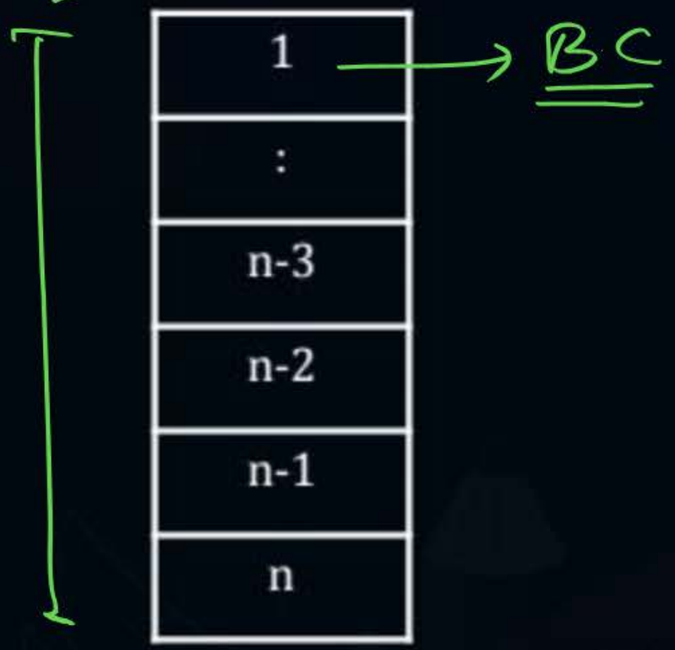
Topic : Space Complexity

TC $\Rightarrow$ $T(n) = T(n-1) + a$

$\Rightarrow O(n)$

Space complexity = $O(n)$

Recursion Stack



BC

Recursion → Height of Recursion Stack

SC: $O(n)$

Auxiliary Space

Recursion stack

Space complexity

The max size of recursion stack at any time during recursion.



Topic : Analysis of Algorithms

#Q.

```
Algo AJ(n)
{
    if (n==1)
        return,
    else
        return AJ(n/2)
}
```

SC = ?

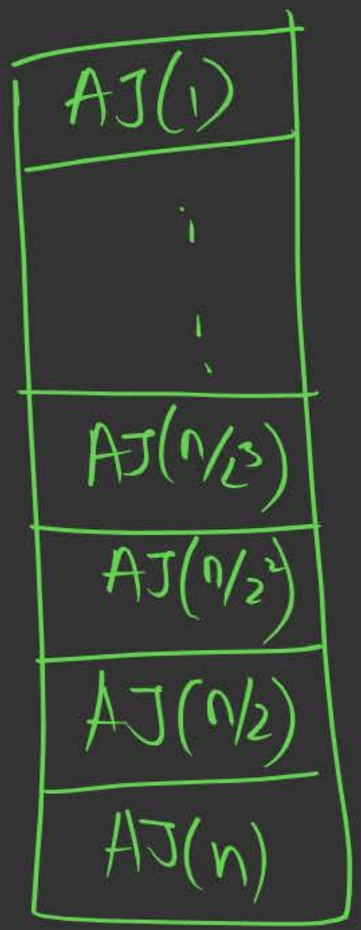
47%

A. O(n)

B. $O(n^2)$

C. O(n logn)

D. O(logn)



$K =$

$SC = O(\log_2 n)$

$n \to n/2 \to n/2^2 \cdots \to$ K th term $= 1$

$n/2^K = 1$

$2^K = n$

$K = \log_2 n$



Topic : Analysis of Algorithms

#Q. Algo AJ(n)

PYQ

TC = ?

```
{
    for (i=1, i<=n;i=i+1)
    {
        for (j=1; j<=n; j=j+i)
        {
            print (i,j)
        }
    }
}
```

nested dependent loop

75%

- A. O(n)
- B. O(n^2)
- C. O(n logn) → 32.5%
- D. O(logn)

$j = j + i$

$i = 1$: $j : 1 \xrightarrow[+1]{} n \longrightarrow n$

$i = 2$: $j : 1 \xrightarrow[+2]{} n \longrightarrow n/2$

$i = 3$: $j : 1 \xrightarrow[+3]{} n \longrightarrow n/3$

$i = 4$: $j : 1 \xrightarrow[+4]{} n \longrightarrow n/4$

$i = n$: $j 1 \xrightarrow[+n]{} n \longrightarrow 1$

$n + n/2 + n/3 + n/4 \cdots n/n$

$= n \underbrace{\left[\frac{1}{1} + \frac{1}{2} + \frac{1}{3} \cdots \frac{1}{n} \right]}_{\log n}$

$= \underline{(n * \log n)}$



Topic : Analysis of Algorithms

PYQ

#Q.
```
Algo AJ(n)
{
    int i, j ,k = 0
    for (i=n/2; i <=n;i =i+1)
    {
        for (j=2; j <=n; j = j*2)
        {
            k= k+n/2
        }
    }
    return (k)
}
```

$n/2 \rightarrow n : O(n)$

$\longrightarrow O(\log n)$

(Q) The value returned by Algo is $O(\ )$?

A. $O(n)$

B. $O(n^2$ [illegible] $)$

C. $O(n \log n)$ → TC

D. $O(n^2 \log n)$

outer $\longrightarrow O(n)$

inner $\longrightarrow O(\log_2 n)$

Overall TC $\longrightarrow O(n \times \log_2 n)$

Value of k returned?

$\longrightarrow$ $n/2$ is added every time the loop runs.

Overall, $k = (n \times \log n) * n/2$ $\quad$ sum $\longrightarrow n/\log n$

$= O(n^2 \log n)$



Topic : Analysis of Algorithms

#Q. Consider functions function_1 and function_2 expressed in pseudocode as follow:

Function_1	Function_2

While n > 1 do for i = 1 to n do
x = x + 1;
end for
n = [n/2];
end while
	for i = 1 to 100*n do
	x = x + 1;
	end for

Let $f_1(n)$ and $f_2(n)$ denote the number of times the statement "x=x+1" is executed in function_1 and function_2, respectively. Which of the following statement is/are TRUE?

A $f_1(n) \in \theta(f_2(n))$

B $f_1(n) \in 0(f_2(n))$

C $f_1(n) \in \omega(f_2(n))$

D $f_1(n) \in 0(n)$



THANK - YOU